

20260604-4-6교시

날짜: 2026/06/04 12:27

개요

1. 원본 노트 (손글씨 포함)..... 2

1. 원본 노트 (손글씨 포함)

회의록 (강의형 수업) — 2026-06-04

1) 기본 정보

- 일사: 2026-06-04
- 형태: 대면+비대면 혼합(주피터랩/VS Code 사용)
- 주요 진행자: 강사(성함 확인 필요)
- 참석자: 수강생 다수(예: 승현 언급)

2) 주요 안건 및 논의 내용

2.1 파이썬 라이브러리 개념 정리

- 라이브러리 정의: 재사용 가능한 코드 집합(개발자가 만들고 사용자는 가져다 씀).
- 기본 문법만으로는 시각화/크롤링/AI 등 고급 기능 구현이 제한적이므로 **라이브러리 활용이 필수**.
- 도서관 비유:
 - 필요한 책만 골라 빌리듯, 필요한 라이브러리만 선택해 사용.

2.2 모듈 • 패키지 • 라이브러리 • 프레임워크 차이

- **모듈(Module)**: 단일 파이썬 파일(함수/클래스/변수 정의).
- **패키지(Package)**: 여러 모듈을 폴더로 묶어 관리.
 - 폴더에 `__init__.py`(초기화 파일) 필요.
- **라이브러리(Library)**: 패키지를 포함한 더 큰 배포 단위(문서/매뉴얼 포함, “상품화된 코드”).
- **프레임워크(Framework)**: 더 큰 규모의 공통 개발 구조(여러 사람이 공동 개발/기여).
 - 프레임워크에 **기여(소스코드 PR)** 하는 것은 큰 스펙으로 언급.

2.3 라이브러리 선택 기준(탐색 시 주의점)

- GitHub/PyPI에서 확인할 항목:
 - **유지보수성**: 최근 업데이트 여부(최소 6개월 이내 업데이트 권장)
 - **호환성**: 파이썬 버전/의존성 충돌 가능성
- “나만 아는 라이브러리” 보다 **대중적이고 활발한 프로젝트** 권장.
- 개발 커뮤니티(오픈카톡 등)에서 **현업 사용 여부** 질의는 도움이 되나,
 - 지식 없이 무작정 질문하면 답을 검증하기 어려우므로 주의.

2.4 표준 라이브러리 vs 외부 라이브러리, PyPI 소개

- 파이썬 설치 시 제공되는 **표준 라이브러리** 존재.
- 추가 설치가 필요한 **외부 라이브러리**는 PyPI에서 관리됨.
- 개발자 지망자는 PyPI 회원가입 권장(필수는 아님).

2.5 가상환경(venv) 필요성과 혼란 포인트

- 가상환경 목적: **프로젝트별 의존성/버전 충돌 방지**.
- 현실적 결론(강사 의견):
 - 입문자에게는 관리가 어렵고 혼란 유발.
 - 여러 목적(웹/분석/AI)을 **동시에** 하는 경우에 특히 필요하지만,
 - 대부분은 한 목적 중심이므로 **가상환경이 필수는 아님**.
- 실습 데모(터미널):
 - 가상환경 **activate/deactivate** 후 which python 경로가 달라짐.
 - 동일 라이브러리라도 **가상환경 안/밖 설치 위치가 달라져** “설치했는데 import 안됨” 상황이 발생.
- 설치 속도 비교:
 - pip install -r requirements.txt 방식은 **느릴 수 있음**(구형 PC에서 수십 분 소요 사례 언급).
 - **uv 사용 경험상 더 빠르고 편리**하다고 강조.

2.6 라이브러리 설치/사용 실습 포인트

- 설치 방식:
 - requirements.txt 사용 예시 제시
 - uv 기반 환경(pyproject.toml, uv sync)도 언급
- 사용 시 권장:
 - 라이브러리 import 후 **버전 확인**(문서/검색/질문 시 필요)
- 예시:
 - `import pandas as pd; print(pd.__version__)`
 - `import matplotlib as mpl; print(mpl.__version__)`
- 버전 중요 이유:
 - 문서/답변/AI가 특정 버전을 기준으로 안내할 수 있어 **경고/오작동** 발생 가능.

2.7 오픈소스와 “배포” 의 의미(개발 생태계 설명)

- 왜 코드를 공개하는가:
 - **커뮤니티 기여**
 - **포트폴리오/명성 구축(취업 경쟁력)**
- 핵심 메시지:
 - “회사=학교가 아님” → 취업 시 “잘 배우겠다” 만으로는 부족
 - 개발 분야에서 **오픈소스 기여가 가장 확실한 PR/검증 근거**가 될 수 있음.
- GitHub Issues를 통한 생태계 예시:
 - 기능 제안/버그 제보 → 개발자가 대응 → 라이브러리 발전
- 배포(패키징) 흐름 개요:
 - pyproject.toml에 **빌드 시스템/작성자/URL 등 메타정보** 작성
 - src/패키지명/`__init__.py` 구조로 패키지 구성
 - **pytest 테스트 코드 작성 및 통과** 후 배포 진행
- 강사 데모 제한:
 - PyPI 로그인 인증 메일 문제로 **실제 배포 시연은 실패**(“확인 필요”)
 - 대신 로컬 구성/흐름 설명 중심으로 진행

3) 결정 사항

- 수업 운영 측면:
 - 라이브러리 설치/활용 중심으로 다음 단계 진행.
 - 가상환경은 개념을 이해하되, 실무/학습 효율 관점에서 **uv 기반 흐름을 우선**(강사 권장).

4) 액션 아이템(과제/요청)

4.1 수강생 공통

- 주피터랩에서 라이브러리 import 후 **버전 출력 실습** 진행.
- 공식 문서 확인 습관화(버전 기반으로 조회).

4.2 개발자 지망 수강생

- (권장) PyPI 회원가입 및 기본 설정 진행(**필요 시**).
- 개인 포트폴리오용으로 **작은 라이브러리 1개 제작/배포 도전** 권장:
 - 패키지 구조 구성(src/, `__init__.py`)
 - pytest 테스트 작성 및 통과
 - 버전 업데이트(0.0.1 → 0.0.2 ...) 방식으로 점진적 발전

4.3 다음 수업 준비

- **개발환경 세팅을 신속히 완료**할 수 있도록 사전 연습 요청(다음날 10분 내 완료 목표 언급).

5) 이슈 및 확인 필요

- 강사 PyPI 계정 로그인 시 **새 디바이스 인증 메일** 미수신으로 배포 시연 불가
- **원인/해결 확인 필요**(메일/스팸/보안설정 등).
- 특정 “유명 쇼핑몰” 의 구형 파이썬 버전 사용 사례 언급(회사명은 비공개).

6) 마무리

- 파이썬 기초 문법 파트는 사실상 마무리 단계이며,
- 이후 수업은 **라이브러리를 가져다 쓰는 실전 활용 중심**으로 전환 예정.